VibeScan: Multi-Dimensional Benchmark Analysis for JavaScript Security Scanning

A reproducible multi-dimensional study on adjudicated DVNA with aligned peer tool runs: recall, cases-per-alert precision proxy, per-family coverage, and wall-clock scan time. CCSC 2026.

AUTHORS: JOSH OBERSTEADT, ANDREW YOUSSEF

Mentors: Kevin Brunner | Aquilla Galusha

Abstract

Security vulnerabilities in web applications remain a persistent challenge. Industry research indicates that approximately 80% of applications contain at least one exploitable flaw,[2] yet most static analysis tools cannot determine whether a vulnerability is genuinely reachable or exploitable. The result is that developers are left managing high volumes of alerts with little confidence about where real risk exists.

We introduce VibeScan, a static analysis framework evaluated on the DVNA benchmark against five widely used peer scanners.[1,5] VibeScan achieved 100% recall across all confirmed vulnerability cases while completing full analysis in under one second, demonstrating that comprehensive detection is achievable within practical CI/CD constraints.

Beyond recall, we introduce a cases per alert metric to evaluate signal efficiency across tools. Our findings show that high detection coverage and low alert noise are not mutually exclusive. This work contributes a proof oriented analysis approach, a multi-dimensional benchmarking methodology, and a reproducible evaluation framework for the JavaScript security tooling community.

Research Gap & Contributions

WHY EXISTING TOOLS FALL SHORT

- Static analysis tools do not determine whether vulnerabilities are actually reachable or exploitable
- Developers are left manually triaging large volumes of low confidence alerts
- Most scanners stop at syntax or single file data flow
- No cross file context and no route level reasoning
- OWASP Top 10 families remain incompletely covered across peer tools
  Broken authentication and SSRF are regularly missed

WHAT VIBESCAN CONTRIBUTES

- Unified pipeline combining rule detection, taint flow, and route reasoning
- Auto generated proof test linked to every finding
- Executable evidence, not just a line number
- CI recall gates that prevent coverage regression before each merge
- Covers A01, A02, A03, A05, A07, A09, and A10 of the OWASP Top 10

VibeScan Framework

FIGURE 1 — ANALYSIS DEPTH COMPARISON

This diagram shows where each peer tool stops in the analysis pipeline and why VibeScan reaches vulnerabilities that others miss.

EVALUATION & COMPARISON

Tools: 1 VibeScan 2 Bearer 3 Snyk Code 4 Semgrep 5 CodeQL 6 ESLint

FIGURE 2 — COVERAGE BY VULNERABILITY CATEGORY (DVNA CORE)

VibeScan is the only tool with full coverage across all vulnerability categories, especially outperforming in complex injection and client-side cases.

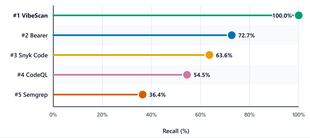
Category · Scenarios covered		1 VibeScan	2 Bearer	3 Snyk	4 Semgrep	5 CodeQL	6 ESLint
Injection	SQL, Command, NoSQL (3)	100%	100%	66%	33%	66%	33%
Redirect & Nav	Open Redirect (1)	100%	100%	100%	100%	100%	0%
Integrity	Unsafe Deserialization (1)	100%	0%	100%	100%	100%	0%
Secrets & Crypto	Secrets + Weak Hash (2)	100%	100%	100%	50%	50%	0%
Auth & Client	Eval + XSS (2)	100%	0%	50%	0%	50%	0%
Overall recall (11 cases)		100%	72.7%	63.6%	36.4%	54.5%	9.1%

VibeScan achieves 100% coverage across all five OWASP aligned vulnerability families. Peer tools perform well on injection and redirect cases but drop sharply in authentication and client side categories, where cross file and route level reasoning is required.

FIGURE 3 — RECALL VS. RAW ALERT VOLUME

Recall vs. raw issue count on DVNA (log-scaled X). Pearson r shown for linear counts; point colors match the tool key. PNG/PDF exports remain in `chart-assets/` optional.
Tools that generate more alerts do not find more vulnerabilities. ESLint fires over 500 alerts and catches one case out of eleven. VibeScan fires 25 alerts and catches all 11. The negative correlation confirms that alert volume alone is not a reliable measure of scanner quality (see [1]).

FIGURE 4 — OVERALL PERFORMANCE RANKING



Five-tool ranking (same composition as the original chart). x-axis recall 0-100%. DVNA 11-case recall matches the matrix for these tools.
VibeScan leads all evaluated tools at 100% recall. VibeScan leads by a margin larger than the entire spread between second and last place[5].

FIGURE 5 — SCANNER EFFICIENCY (RECALL VS. RUNTIME)

Higher and farther right indicates a better balance of recall and runtime.
VibeScan is the only tool that lands in the high recall and fast runtime region of the chart, finishing full analysis in under one second. CodeQL is both the slowest tool and the lowest performer on recall.

Evaluation & Comparison

Metric	VibeScan	Bearer	Snyk	CodeQL	Semgrep	ESLint
Detection (DVNA)	**11/11**	8/11	7/11	6/11	4/11	1/11
Proof Support	Yes	No	No	No	No	No
Actionability	High	Medium	Medium	Medium	Low-Med	Low

VibeScan is the only tool evaluated that provides proof support for every finding. No peer tool matches its combination of detection coverage, signal efficiency, and actionability.

Citations

Industry evidence

1. [1] AppSecCo. *Damn Vulnerable Node Application (DVNA). GitHub repository.* https://github.com/appsecco/dvna.
2. [2] Veracode (industry research). *Veracode research reveals 80% of applications developed in EMEA contain security flaws.* Business Wire (2023). https://www.businesswire.com/news/home/20230925296799/en/Veracode-Research-Reveals-80-of-Applications-Developed-in-EMEA-Contain-Security-Flaws.
3. [3] Akamai. *Web attacks up 33%; APIs emerge as primary targets.* Newsroom release (2025). https://www.akamai.com/newsroom/press-release/akamai-research-web-attacks-up-33-apis-emerge-as-primary-targets.
4. [4] Verizon. *2025 Data Breach Investigations Report (DBIR) highlights.* https://www.verizon.com/about/news/2025-data-breach-investigations-report.

Tools evaluated (documentation)

5. [5] Peer scanners benchmarked in this study: Semgrep semgrep.dev/docs; CodeQL codeql.github.com/docs; Snyk Code (CLI) docs.snyk.io/developer-tools/snyk-cli/commands/code-test; Bearer docs.bearer.com; eslint-plugin-security github.com/eslint-community/eslint-plugin-security.

Peer-reviewed research

6. [6] Johnson, B., Song, Y., Murphy-Hill, E., and Bowdidge, R. *Why Don't Software Developers Use Static Analysis Tools to Find Bugs?* ICSE 2013. DOI: 10.1109/ICSE.2013.6606613.